SRM Institute of Science and Technology

(Deemed to be University U/S 3 of UGC Act, 1956)

Faculty of Science and Humanities

Tiruchirappalli, Tamil Nadu-621 105

Department of Computer Applications

PRACTICAL RECORD

NAME :

REGISTER NO : RA25322420500

COURSE : MCA(GEN-AI)

SEMESTER/ YEAR : II / I

SUBJECT CODE : PCA25C07J

SUBJECT NAME : ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

April 2026

SRM Institute of Science and Technology

(Deemed to be University U/S 3 of UGC Act, 1956)

Faculty of Science and Humanities

Tiruchirappalli, Tamil Nadu-621 105

Department of Computer Applications

BONAFIDE CERTIFICATE

This is to certify that the bonafide work is done by Mr/Ms. _____ Register No.RA25322420500____ in **Artificial Intelligence and Machine Learning (Subject Code: PCA25C07J)** at Computer Lab, SRMIST, Tiruchirappalli, in April 2026.

STAFF IN-CHARGE **HEAD OF THE DEPARTMENT**

Submitted for the University Practical Examination held at SRMIST, Tiruchirappalli, Department of Computer Applications on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

INDEX

S. No.	Date	Name of the Program	Page No.	Signature
1		Xor Multi-Layer Perceptron With User Input	1	
2		Single Neuron Perceptron For And Gate	5	
3		Simple Ann With Sigmoid For Binary Classification	8	
4		Weight And Bias Effects On Neuron Output	10	
5		Depth First Search (Dfs)	12	
6		Breadth First Search (Bfs)	14	
7		A* Algorithm	16	
8		Ao* Algorithm	18	
9		Greedy Best First Search	20	
10		K-Means Clustering In Prolog	22	
11		Hierarchical Clustering In Prolog	24	

Program 1: XOR Multi-Layer Perceptron with User Input

Aim:

To implement an **Artificial Neural Network (ANN)** using backpropagation to solve the non-linearly separable **XOR gate** problem and predict outputs based on user input.

Algorithm Steps:

1. Define the **sigmoid activation function** and its derivative to handle non-linearity.
2. Initialize the training data for the XOR logic (Inputs: ,; Labels:).
3. Define a network structure with **2 input neurons, 4 hidden neurons, and 1 output neuron**.
4. Initialize the weights for input-to-hidden and hidden-to-output layers with random values.
5. Set the learning rate (0.1) and number of epochs (10,000).
6. **Training Loop:**
   - **Forward Pass:** Calculate the dot product of inputs and weights, then apply the sigmoid function.
   - **Error Calculation:** Subtract the predicted output from the actual target.
   - **Backpropagation:** Calculate gradients using the sigmoid derivative and update weights to minimize error.
7. Prompt the user to enter two binary inputs (0 or 1).
8. Pass the user input through the trained network and apply a **0.5 threshold** to determine the final predicted output (0 or 1).

Programme:

```
import numpy as np
```

```
# Sigmoid activation function
def sigmoid(x):
return 1 / (1 + np.exp(-x))

# Derivative of sigmoid
def sigmoid_derivative(x):
return x * (1 - x)

# XOR training data
X = np.array([[0,0],
[0,1],
[1,0],
[1,1]])

y = np.array([[0],
[1],
[1],
[0]])

np.random.seed(1)

# Network structure
input_neurons = 2
hidden_neurons = 4
output_neurons = 1

weights_input_hidden = np.random.uniform(size=(input_neurons, hidden_neurons))
weights_hidden_output = np.random.uniform(size=(hidden_neurons,
output_neurons))

learning_rate = 0.1
epochs = 10000

# Training
```

```
for epoch in range(epochs):

hidden_layer = sigmoid(np.dot(X, weights_input_hidden))
output_layer = sigmoid(np.dot(hidden_layer, weights_hidden_output))

error = y - output_layer
d_output = error * sigmoid_derivative(output_layer)

error_hidden = d_output.dot(weights_hidden_output.T)
d_hidden = error_hidden * sigmoid_derivative(hidden_layer)

weights_hidden_output += hidden_layer.T.dot(d_output) * learning_rate
weights_input_hidden += X.T.dot(d_hidden) * learning_rate

print("Training completed!")

# Taking user input
a = int(input("Enter first input (0 or 1): "))
b = int(input("Enter second input (0 or 1): "))

user_input = np.array([[a, b]])

# Forward pass for user input
hidden_layer = sigmoid(np.dot(user_input, weights_input_hidden))
output = sigmoid(np.dot(hidden_layer, weights_hidden_output))

print("Raw Output:", output)

# Convert to 0 or 1
if output >= 0.5:
print("Predicted XOR Output: 1")
else:
print("Predicted XOR Output: 0")
```

Output:

The system prints "Training completed!", prompts for inputs, and displays the raw probability and the final predicted class.

Result:

The multi-layer network successfully learns the XOR pattern, which a single-layer perceptron cannot do.

Program 2: Single Neuron Perceptron for AND Gate

Aim:

To implement a basic **single-neuron Perceptron** to learn the logic of an **AND gate**.

Algorithm Steps:

1. Define a **step function** as the activation method (returns 1 if input ≥ 0, else 0).
2. Set up the AND gate training data (inputs and targets).
3. Initialize weights and bias to zero.
4. Set the learning rate (0.1) and number of epochs.
5. **Training:** For each input, calculate the linear sum ($X \cdot W + bias$), apply the step function, and update weights/bias based on the error (target - prediction).
6. **Testing:** Pass all AND gate inputs through the trained neuron to verify correctness.

Programme:

```
import numpy as np


# Step function

def step(x):

    return 1 if x >= 0 else 0


# Training data for AND gate

X = np.array([[0,0],

              [0,1],

              [1,0],

              [1,1]])
```

```
y = np.array([0,0,0,1])

# Initialize weights and bias
weights = np.zeros(2)
bias = 0
learning_rate = 0.1

# Training
for epoch in range(10):
    for i in range(len(X)):
        linear_output = np.dot(X[i], weights) + bias
        prediction = step(linear_output)
        error = y[i] - prediction

        weights += learning_rate * error * X[i]
        bias += learning_rate * error

print("Training completed!")

# Testing
for i in range(len(X)):
    result = step(np.dot(X[i], weights) + bias)
    print(X[i], "->", result)
```

Output:

"Training completed!" followed by a list of inputs and their mapped outputs (e.g., -> 1).

Result:

The Perceptron converges on the weights necessary to correctly classify AND gate inputs.

Program 3: Simple ANN with Sigmoid for Binary Classification

Aim:

To demonstrate a simple **binary classification** using a single-neuron ANN with **sigmoid activation**.

Algorithm Steps:

1. Define the **sigmoid function** and its derivative.
2. Provide a simple dataset where input 0 maps to 0 and input 1 maps to 1.
3. Initialize random weights and set a learning rate of 0.5.
4. **Training Loop:** Compute the output, determine the error, and update the weights using the gradient (error multiplied by the sigmoid derivative).
5. Print the final probability output for the inputs.

Programme:

```
import numpy as np

def sigmoid(x):
return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
return x * (1 - x)

# Input data
X = np.array([[0],
[1]])

# Output data
y = np.array([[0],
[1]])

np.random.seed(1)
```

```
weights = np.random.rand(1,1)

learning_rate = 0.5

# Training
for epoch in range(1000):
output = sigmoid(np.dot(X, weights))
error = y - output
weights += np.dot(X.T, error * sigmoid_derivative(output)) * learning_rate

print("Final Output:")
print(sigmoid(np.dot(X, weights)))
```

Output:

The final sigmoid activation values for the input data, which should be very close to 0 and 1.

Result:

The program illustrates how gradient descent minimizes error in a continuous activation model.

Program 4: Weight and Bias Effects on Neuron Output

Aim:

To observe how manual adjustments to **weight (*w*)** and **bias (*b*)** influence the final output of a sigmoid-activated neuron.

Algorithm Steps:

1. Define the **sigmoid activation function**.
2. Capture user input for a single data point (x), a weight (w), and a bias (b).
3. Calculate the **weighted sum** ($z=x \cdot w+b$).
4. Apply the sigmoid function to z to get the final activation value.
5. Display the intermediate calculation (z) and the final output.

Programme:

```
import numpy as np

# Sigmoid activation function
def sigmoid(x):
return 1 / (1 + np.exp(-x))

# Input value
x = float(input("Enter input value x: "))

# Taking weight and bias from user
w = float(input("Enter weight w: "))
b = float(input("Enter bias b: "))

# Neuron output
z = x * w + b
output = sigmoid(z)

print(f"\nNeuron calculation: z = x*w + b = {z}")
print(f"Output after sigmoid activation: {output:.4f}")
```

Output:

Displays the mathematical breakdown of the neuron's calculation and the final decimal output (0 to 1 range).

Result:

This exercise confirms that the bias shifts the activation function while the weight changes the strength of the input's influence.

Program 5: Depth First Search (DFS)

AIM:

To implement Depth First Search (DFS) using recursion to traverse a graph represented as an adjacency list.

Algorithm:

1. **Initialize** a set to keep track of visited nodes.
2. **Define** a recursive function dfs(node).
3. **Check** if the current node has been visited. If not, mark it as visited and print it.
4. **Iterate** through each neighbor of the current node in the graph and recursively call the dfs function for each unvisited neighbor.
5. **Start** the traversal from a designated starting node.

Programme:

```
# Depth First Search using recursion
# Graph represented as an adjacency list
graph = {'A': ['B', 'C'],'B': ['D', 'E'],'C': ['F'],'D': [],'E': ['F'],'F': []}
visited = set() # To keep track of visited nodes

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)

# Starting node
start_node = 'A'
print("Depth First Search Traversal:")
dfs(start_node)
```

Output:

Depth First Search Traversal: A B D E F C

Result:

The graph was successfully traversed using the Depth First Search algorithm.

Program 6: Breadth First Search (BFS)

AIM:

To implement Breadth First Search (BFS) using a queue to traverse a graph level by level.

Algorithm:

1. **Initialize** a set for visited nodes and a queue for the traversal.
2. **Add** the starting node to the visited set and the queue.
3. **While** the queue is not empty, pop the leftmost node from the queue and print it.
4. **For each neighbor** of the current node that has not been visited, mark it as visited and add it to the queue.
5. **Repeat** until the queue is empty.

Programme:

```
# Breadth First Search using Queue
from collections import deque

# Graph represented as an adjacency list
graph = {'A': ['B', 'C'],'B': ['D', 'E'],'C': ['F'],'D': [],'E': ['F'],'F': []}
visited = set() # To keep track of visited nodes
queue = deque() # Queue for BFS

def bfs(start):
    visited.add(start)
    queue.append(start)
    while queue:
        node = queue.popleft()
        print(node, end=" ")
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Starting node
```

```
print("Breadth First Search Traversal:")
bfs('A')
```

Output:

```
Breadth First Search Traversal: A B C D E F
```

Result:

The graph was successfully traversed using the Breadth First Search algorithm.

Program 7: A* Algorithm

AIM:

To implement the A* Search algorithm to find the optimal path in a weighted graph using heuristic values.

Algorithm:

1. **Initialize** an open_list with the start node and an empty closed_list. Set the initial cost g of the start node to 0.
2. **Select** the node n from the open_list that has the lowest total estimated cost f(n) = g(n) + h(n), where h(n) is the heuristic value.
3. **If** the selected node is the goal, reconstruct and print the path from start to goal.
4. **Expand** the neighbors of n. For each neighbor, calculate its cost. If it's a better path than previously found, update its cost and parent, and ensure it is in the open_list.
5. **Move** node n from the open_list to the closed_list.

Programme:

```
# A* Algorithm in Python
# Graph with cost
graph = {'A': {'B': 1, 'C': 3},'B': {'D': 1, 'E': 4},'C': {'F': 2},'D': {},'E': {'F': 1},'F': {}}
# Heuristic values (estimated cost to goal)
heuristic = {'A': 5,'B': 3,'C': 4,'D': 2,'E': 1,'F': 0}

def a_star(start, goal):
    open_list = [start]
    closed_list = []
    g = {start: 0}
    parent = {start: start}
    while open_list:
        n = None
        for v in open_list:
            if n is None or g[v] + heuristic[v] < g[n] + heuristic[n]:
                n = v
        if n is None:
            print("Path does not exist")
```

```
            return
        if n == goal:
            path = []
            while parent[n] != n:
                path.append(n)
                n = parent[n]
            path.append(start)
            path.reverse()
            print("Path found:", path)
            return
        for (m, weight) in graph[n].items():
            if m not in open_list and m not in closed_list:
                open_list.append(m)
                parent[m] = n
                g[m] = g[n] + weight
            else:
                if g[m] > g[n] + weight:
                    g[m] = g[n] + weight
                    parent[m] = n
                    if m in closed_list:
                        closed_list.remove(m)
                        open_list.append(m)
        open_list.remove(n)
        closed_list.append(n)
    print("Path does not exist")

# Run the algorithm
a_star('A', 'F')
```

Output:

Path found: ['A', 'B', 'E', 'F']

Result:

The A* algorithm successfully found the optimal path to the goal.

Program 8: AO* Algorithm

AIM:

To implement the AO* algorithm to find the minimum cost to reach a goal in an AND-OR graph.

Algorithm:

1. **Define** an AND-OR graph structure and a set of heuristic values for each node.
2. **Define** a recursive function ao_star(node) to compute the cost of the optimal solution sub-graph.
3. **If** the node is a terminal node, return its heuristic value.
4. **For OR nodes**, calculate the cost of each child branch and select the one with the minimum cost.
5. **For AND nodes**, sum the costs of all children in that branch.
6. **Update** the node's heuristic value with the calculated minimum cost and return it.

Programme:

```
# Simple AO* Algorithm Example
# Heuristic values
H = {'A': 1,'B': 6,'C': 2,'D': 12,'E': 2,'F': 1,'G': 5,'H': 7,'I': 7,'J': 1}
# AND-OR graph structure
Graph = {'A': [['B', 'C'], ['D']],'B': [['E'], ['F']],'C': [['G'], ['H']],'D': [['I'], ['J']]}

def ao_star(node):
    if node not in Graph:
        return H[node]
    min_cost = float('inf')
    for option in Graph[node]:
        cost = 0
        for child in option:
            cost += ao_star(child)
        if cost < min_cost:
            min_cost = cost
    H[node] = min_cost
```

```
    return H[node]

start = 'A'
result = ao_star(start)
print("Minimum cost from node", start, "=", result)
```

Output:

Minimum cost from node A = 3

Result:

The AO* algorithm correctly calculated the minimum cost for the AND-OR graph.

Program 9: Greedy Best First Search

AIM:

To implement Greedy Best First Search to find a goal node based solely on heuristic estimates.

Algorithm:

1. **Initialize** a queue with the starting node.
2. **While** the queue is not empty, select and remove the node with the lowest heuristic value.
3. **Mark** the node as visited and print it.
4. **If** the current node is the goal, terminate the search.
5. **Add** all unvisited neighbors of the current node to the queue and repeat.

Programme:

```
# Greedy Best First Search in Python
# Graph structure
graph = {'A': ['B', 'C'],'B': ['D', 'E'],'C': ['F'],'D': [],'E': [],'F': []}
# Heuristic values (estimated distance to goal)
heuristic = {'A': 5,'B': 3,'C': 4,'D': 2,'E': 1,'F': 0}
visited = []

def greedy_best_first(start, goal):
    queue = [start]
    while queue:
        # choose node with smallest heuristic value
        node = min(queue, key=lambda x: heuristic[x])
        queue.remove(node)
        print(node, end=" ")
        visited.append(node)
        if node == goal:
            print("\nGoal reached!")
            return
        for neighbor in graph[node]:
            if neighbor not in visited:
```

```
            queue.append(neighbor)

# Run the algorithm
print("Greedy Best First Search Traversal:")
greedy_best_first('A', 'F')
```

Output:

Greedy Best First Search Traversal: A C F Goal reached!

Result:

The goal was successfully reached using Greedy Best First Search.

Program 10: K-Means Clustering in Prolog

AIM:

To implement K-Means clustering in Prolog for 2D points and assign a point to the nearest cluster centroid.

Algorithm:

1. Define a **distance function** to calculate Euclidean distance between two points.
2. Define a **nearest function** to compare distances to two centroids and decide which is closer.
3. Define the **kmeans function** to assign a point to the nearest centroid.
4. Load the program in **SWI-Prolog**.
5. Query the program to assign a point to a cluster.

Prolog Program

```
% Calculate distance between two points
distance((X1,Y1),(X2,Y2),D) :-
    DX is X1 - X2,
    DY is Y1 - Y2,
    D is sqrt(DX*DX + DY*DY).

% Find nearest centroid
nearest((X,Y),(C1X,C1Y),(C2X,C2Y),Cluster) :-
    distance((X,Y),(C1X,C1Y),D1),
    distance((X,Y),(C2X,C2Y),D2),
    (D1 =< D2 -> Cluster = 1 ; Cluster = 2).

% Assign point to nearest cluster
kmeans(Point,C1,C2,Cluster) :-
    nearest(Point,C1,C2,Cluster).
```

How to Run in Prolog

1. **Save the Program**
   - Save the above code as kmeans.pl.
2. **Load the Program in SWI-Prolog**

   consult('kmeans.pl').

   OR

   [kmeans].

   Output should be:

   true.

3. **Run the Query**

   kmeans((2,3),(1,1),(5,6),C).

   Output:

   C = 1.

   This means the point (2,3) belongs to **Cluster 1**.

4. **Exit Prolog**

   halt.

Result:

The point is successfully assigned to the nearest cluster using the K-Means logic implemented in Prolog.

Program 11: Hierarchical Clustering in Prolog

AIM:

To implement Hierarchical Clustering in Prolog by finding the closest pair of points from a set of 2D points.

Algorithm:

1. Define a **distance function** to calculate the Euclidean distance between two points.
2. Define a **closest function** to find the two points with the minimum distance in a list of points.
3. Load the program in **SWI-Prolog**.
4. Query the program to find the closest pair of points.

Prolog Program

```
% Distance between two points
distance((X1,Y1),(X2,Y2),D) :-
    DX is X1 - X2,
    DY is Y1 - Y2,
    D is sqrt(DX*DX + DY*DY).

% Find the closest pair of points
closest((A,B),Points) :-
    member(A,Points),
    member(B,Points),
    A \= B,
    distance(A,B,D),
    \+ (member(C,Points), member(D2,Points),
        C \= D2,
        distance(C,D2,D3),
        D3 < D).
```

Example Query

closest(P,[(1,2),(2,3),(5,6),(8,9)]).

Output

P = ((1,2),(2,3)).

This means **(1,2) and (2,3) are the closest points**, so they would be merged first in hierarchical clustering.

Result:

The closest pair of points was successfully identified, demonstrating the first step of Hierarchical Clustering in Prolog.